

```
#include <iostream>

#include <vector>

#include <queue>

#include <set>

#include <map>

using namespace std;

// ----- Graph Input -----

map<int, vector<int>>> createGraph() {

    map<int, vector<int>>> graph;

    int n, e;

    cout << "Enter number of vertices: ";

    cin >> n;

    cout << "Enter number of edges: ";

    cin >> e;

    for (int i = 0; i < n; i++) {

        graph[i] = vector<int>();

    }

    cout << "Enter edges (u v):" << endl;

    for (int i = 0; i < e; i++) {

        int u, v;

        cin >> u >> v;

        graph[u].push_back(v);

        graph[v].push_back(u);

    }
```

```

    return graph;
}

// ----- Tree Printer -----

void printTree(map<int, vector<int>>& tree, int root,
               string indent = "", bool last = true) {

    cout << indent;

    if (last)
        cout << "|__ ";
    else
        cout << "|-- ";

    cout << root << endl;

    indent += (last ? "  " : "| ");

    vector<int> children = tree[root];

    for (int i = 0; i < children.size(); i++) {
        bool lastChild = (i == children.size() - 1);
        printTree(tree, children[i], indent, lastChild);
    }
}

// ----- DFS Helper -----

void dfsHelper(map<int, vector<int>>& graph,
               int node,
               set<int>& visited,
               vector<int>& traversal,

```

```

        map<int, vector<int>>& tree,
        int parent = -1) {

visited.insert(node);
traversal.push_back(node);

if (parent != -1) {
    tree[parent].push_back(node);
}

for (int neighbor : graph[node]) {
    if (visited.find(neighbor) == visited.end()) {
        dfsHelper(graph, neighbor, visited, traversal, tree, node);
    }
}
}

void dfsDiagram(map<int, vector<int>>& graph, int start) {

    set<int> visited;
    vector<int> traversal;
    map<int, vector<int>> tree;

    dfsHelper(graph, start, visited, traversal, tree);

    cout << "\nDFS Traversal Path:" << endl;

    for (int node : traversal) {
        cout << node << " ";
    }
}

```

```

    cout << "\n\nDFS Tree Diagram:" << endl;
    printTree(tree, start);
}

// ----- BFS -----

void bfsDiagram(map<int, vector<int>>& graph, int start) {

    set<int> visited;
    queue<int> q;

    vector<int> traversal;

    map<int, vector<int>> tree;
    map<int, int> levels;

    visited.insert(start);
    q.push(start);
    levels[start] = 0;

    while (!q.empty()) {

        int node = q.front();
        q.pop();

        traversal.push_back(node);

        for (int neighbor : graph[node]) {

            if (visited.find(neighbor) == visited.end()) {

```

```

        visited.insert(neighbor);
        q.push(neighbor);

        tree[node].push_back(neighbor);

        levels[neighbor] = levels[node] + 1;
    }
}

```

```

cout << "\nBFS Traversal Path:" << endl;

```

```

for (int node : traversal) {
    cout << node << " ";
}

```

```

cout << "\n\nBFS Levels:" << endl;

```

```

map<int, vector<int>>> levelMap;

```

```

for (auto x : levels) {
    levelMap[x.second].push_back(x.first);
}

```

```

for (auto x : levelMap) {

```

```

    cout << "Level " << x.first << ": ";

```

```

    for (int node : x.second) {
        cout << node << " ";
    }

```

```

        cout << endl;
    }

    cout << "\nBFS Tree Diagram:" << endl;
    printTree(tree, start);
}

// ----- Main Menu -----

int main() {

    map<int, vector<int>>> graph = createGraph();

    while (true) {

        cout << "\nMENU" << endl;
        cout << "1. DFS Diagram" << endl;
        cout << "2. BFS Diagram" << endl;
        cout << "3. Exit" << endl;

        int choice;
        cout << "Enter choice: ";
        cin >> choice;

        if (choice == 1) {

            int start;
            cout << "Enter starting node: ";
            cin >> start;

            dfsDiagram(graph, start);

```

```
}
```

```
else if (choice == 2) {
```

```
    int start;
```

```
    cout << "Enter starting node: ";
```

```
    cin >> start;
```

```
    bfsDiagram(graph, start);
```

```
}
```

```
else if (choice == 3) {
```

```
    break;
```

```
}
```

```
else {
```

```
    cout << "Invalid choice!" << endl;
```

```
}
```

```
}
```

```
return 0;
```

```
}
```